

Informe Técnico: Evaluación y Selección de TinyLLM (SLM) para Entorno Industrial (Edge AI)

1. Contexto del Proyecto

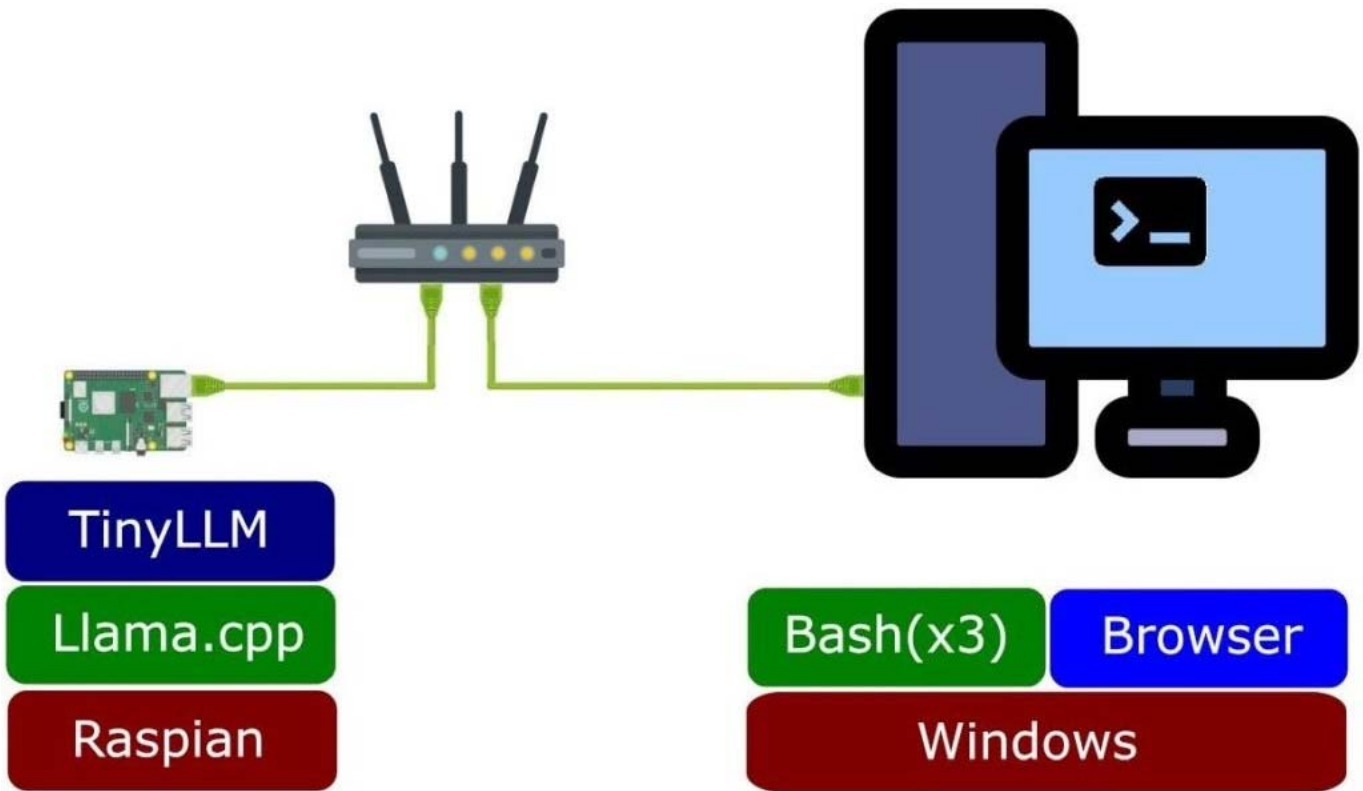
El objetivo es implementar un sistema de Inteligencia Artificial Generativa en un dispositivo **EdgeBox RPi-200 (Raspberry Pi CM4, 4GB RAM)**. El sistema actuará como orquestador en un entorno industrial de pesaje, interactuando con una base de datos MariaDB, una aplicación Python de control, biometría facial y un bot de Telegram.

El desafío principal radica en balancear la **capacidad de razonamiento** (para detección de anomalías en datos gestionados a través de base de datos mariaDB) con las **restricciones de hardware** (latencia de CPU y consumo de memoria).

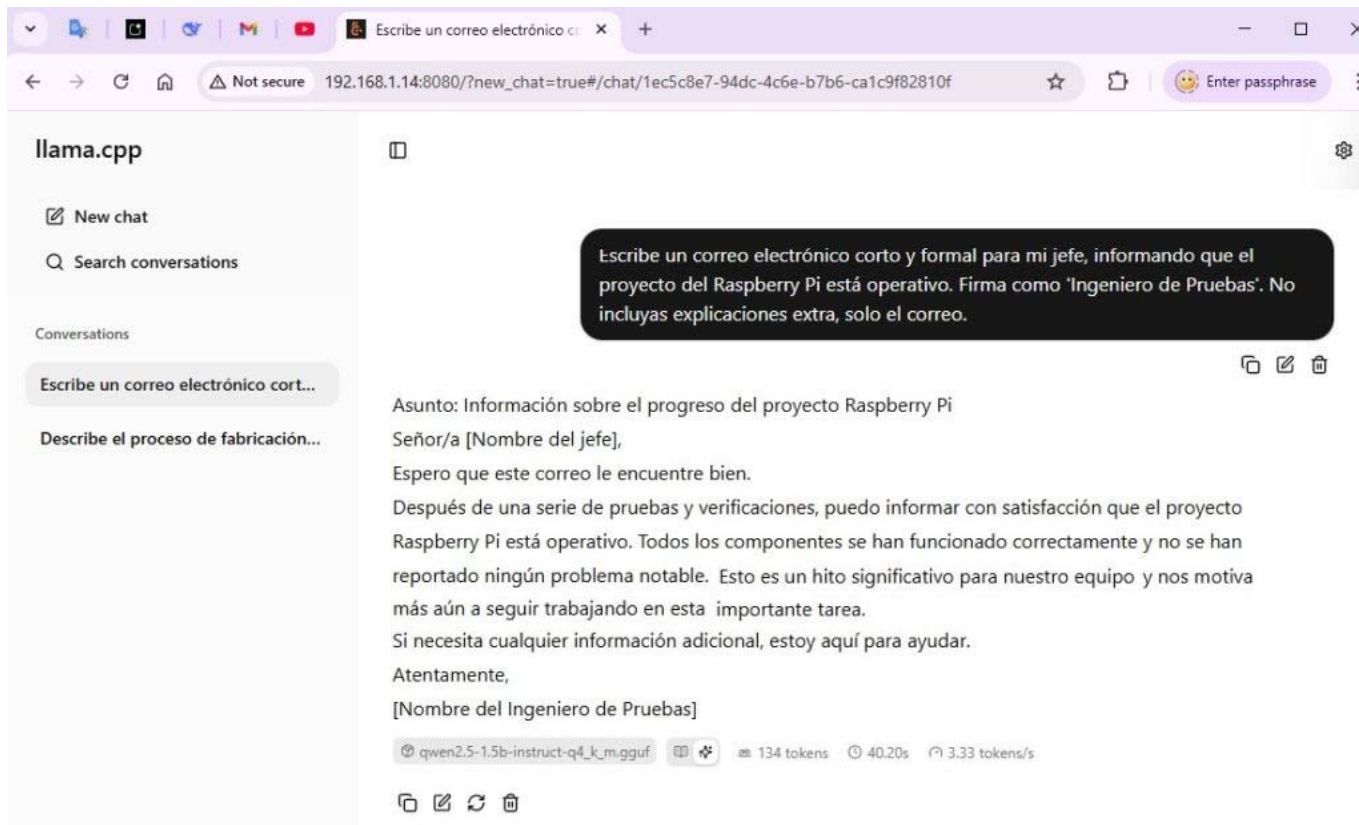
2. Metodología de Evaluación

Para determinar el modelo más adecuado, se diseñó una batería de pruebas estandarizada ejecutada sobre el motor de inferencia `llama.cpp` (compilado específicamente para el sistema de prueba basado en procesador ARM64, que es similar al sistema final).

Se realizó una instalación fresca de Raspberry Pi OS x64 (en memoria eMMC del CM4), con el mínimo de paquetes recomendado, para asegurar un entorno actualizado y evitar paquetes o servicios adicionales que puedan interferir en los resultados o sobrecargar el hardware. Adicionalmente se instaló una unidad SSD del tipo PCIe de 128GB en la carrierboard del CM4, donde se almacenaron los binarios de todos los TinyLLM bajo prueba.



El motor de inferencia corrió en la plataforma señalada y fue accedido vía Web desde otra computadora utilizando un browser, a fin de no cargar de más el sistema en prueba.



Adicionalmente se monitorearon condiciones de temperatura y estabilidad del procesador encontrando que siempre trabajó muy por debajo de su temperatura de seguridad.

- * Temperatura de trabajo: < 63 °C
- * Temperatura de seguridad: 80 °C

```
rojecas@nukak: ~
12:32:26 -> temp=47.7'C | Freq: 1500 MHZ
12:32:28 -> temp=47.7'C | Freq: 1500 MHZ
12:32:30 -> temp=47.2'C | Freq: 1500 MHZ
12:32:32 -> temp=47.7'C | Freq: 1500 MHZ
12:32:34 -> temp=47.2'C | Freq: 1500 MHZ
12:32:36 -> temp=47.2'C | Freq: 1500 MHZ
12:32:38 -> temp=48.2'C | Freq: 1500 MHZ
12:32:40 -> temp=48.2'C | Freq: 1500 MHZ
```

Todos los modelos de lenguaje se cargaron en las mismas condiciones de operacion utilizando el siguiente comando:

```
./build/bin/llama-server -m <ruta_modelo> -c 2048 --host 0.0.0.0 --port 8080
```

```
rojecas@nukak: ~
0[|] 2.0% Tasks: 49, 51 thr, 143 kthr; 2 running
1[|] 1.3% Load average: 0.04 0.57 0.62
2[|] 2.6% Uptime: 00:23:36
3[|] 3.3%
Mem[|] 1.37G/3.70G
Swp[|] 0K/2.00G

Main 1.3%
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
3323 rojecas 20 0 8136 4592 3124 R 1.3 0.1 0:10.36 htop
331 root 20 0 26652 9632 8296 S 0.7 0.2 0:01.59 /usr/lib/systemd/systemd-journald
1089 rojecas 20 0 22932 12224 9780 S 0.7 0.3 0:03.13 /usr/lib/systemd/systemd --user
3287 rojecas 20 0 21644 7140 5160 S 0.7 0.2 0:00.50 sshd-session: rojecas@pts/3
1 root 20 0 25344 14472 10176 S 0.0 0.4 0:04.62 /sbin/init
371 systemd-ti 20 0 92228 7556 6452 S 0.0 0.2 0:00.15 /usr/lib/systemd/systemd-timesyncd
```

donde -c 2048: La bandera -c (context size). Define cuántos "tokens" (piezas de palabras) puede recordar el modelo en la conversación actual.

2.1 Métricas Cuantitativas

Se toman directamente de los valores que informa el motor de inferencia (llama.cpp) al finalizar la misma.

- **Generation Speed (t/s - tokens por segundo):** Velocidad de escritura medida en tokens por segundo. Se considera aceptable > 2.5 t/s para interacción humana fluida.
- **RAM Usage:** Consumo de memoria residente (RSS) del modelo cargado + contexto.
- **Load Time:** Tiempo de carga del modelo en memoria.

2.2 Pruebas Cualitativas (Prompts)

Se sometió cada modelo a 4 pruebas de estrés funcional:

1. **Lógica Matemática (Prueba A):** Problema aritmético secuencial (Manzanas). Evalúa la capacidad de seguir estados.

Prompt: Tengo 5 manzanas. Me como 2 y luego compro 3 más.
¿Cuántas manzanas tengo ahora? Explica paso a paso.

2. **Redacción Corporativa (Prueba B):** Generación de correo formal con instrucciones negativas ("no incluir explicaciones extra").

Prompt: "Escribe un correo electrónico corto y formal para mi jefe, informando que el proyecto del Raspberry Pi está operativo. Firma como 'Ingeniero de Pruebas'. No incluyas explicaciones extra, solo el correo."

3. **Razonamiento Físico (Prueba C):** Pregunta de sentido común (Vaso de agua boca abajo). Evalúa alucinaciones sobre leyes físicas.

Prompt: "Si pongo un vaso de agua lleno boca abajo sobre una mesa, ¿qué pasa cuando levanto el vaso? Responde en una sola frase."

4. **Generación de Código (Prueba D):** Función Python simple. Evalúa sintaxis y lógica de programación.

Prompt: "Escribe una función simple en Python que reciba una lista de números y devuelva solo los números pares."

2.3 TinyLLM bajo Prueba

Los siguientes Grandes Modelos de Lenguaje (LLM por sus siglas en inglés), en versión destilada y cuantizados (comprimidos), fueron escogidos para las pruebas por ser prometedores según los foros especializados en el tema para sistemas de hardware iguales o equivalentes a los utilizados en esta prueba.

1. **Modelo: Llama**, Versión 3.2, Parámetros: 1B
 2. **Modelo: Llama**, Versión 3.2, Parámetros: 3B
 3. **Modelo: Qwen**, Versión 2.5, Parámetros: 1.5B
 4. **Modelo: PHI**, Versión 2, Parámetros: 2.7B
 5. **Modelo: PHI**, Versión 3 mini, Parámetros: 3.8B (es el modelo más pequeño de esta serie)
 6. **Modelo: DeepSeek** Versión: R1 Distill, Parámetros: 1.5B (modelo razonador)
-

3. Datos Relevantes de la Evaluación

A continuación, se presentan los resultados obtenidos en las pruebas de campo.

Modelo	Tamaño	Velocidad (t/s)	RAM (GB)	Lógica (Manzanas)	Física (Vaso)	Email	Código (Python)	Observaciones
Llama 3.2	1B	4.05	1.06	✗ Fallo	✗ Alucinación Grave	✗ Alucinación Grave	✗ Error Lógico	Rápido pero propenso a alucinaciones severas y formato JSON no solicitado.
Llama 3.2	3B	1.54	2.41	✗ Alucinación	✗ Alucinación	✗ Alucinación	✗ Alucinación	Sufrió de "Function Calling Hallucination", respondiendo todo en JSON en lugar de texto natural.
Qwen 2.5	1.5B	3.37	1.32	⚠ Fallo Lectura	✗ Fallo	✓ Perfecto	✓ Excelente	Muy equilibrado. Falló en aritmética por lectura errónea, pero excelente en código y redacción.
Phi-2	2.7B	1.80	2.50	✗ Colapso	✗ Colapso	✗ Colapso	✗ Colapso	Modelo inestable. Generó bucles infinitos de código sin sentido.
Phi-3 Mini	3.8B	1.34	3.29	✓ Perfecto	✓ Perfecto	✓ Perfecto	✓ Excelente	Calidad de respuesta superior ("State of the Art"), pero muy lento para chat fluido.
DeepSeek R1	1.5B	3.26	1.38	✓ Perfecto	✓ Perfecto	⚠ Fallo	✓ Excelente	Razonamiento superior (Chain of Thought). pero muy verboso.

4. Comparativa y Selección Final (Orden de Elegibilidad)

Considerando que el hardware final dispone de **8 GB de RAM**, se ha ajustado la recomendación para maximizar la inteligencia (necesaria para el manejo de herramientas SQL) sin sacrificar la estabilidad del sistema.

🏆 1. Qwen 2.5 - 3B (Instruct) - *La Elección Equilibrada*

Nota: Aunque se probó la versión 1.5B, se espera que para producción (en dispositivo final con 8GB RAM) al escalar a la versión de 3 Billones, se solucionen algunos problemas de lectura.

- **Argumentación:**

- **Capacidad de Herramientas (Tool Use):** La familia Qwen 2.5 es líder en el mercado Open Source en capacidad de "Function Calling". Esto es crítico para la arquitectura propuesta donde el LLM debe decidir cuándo consultar la base de datos MariaDB.
- **Estabilidad de Formato:** A diferencia de Llama 3.2, Qwen respeta estrictamente los formatos JSON cuando se le pide, y habla natural cuando no.
- **Uso de Recursos:** Ocupará aprox. **2.8 GB de RAM**, dejando 5 GB libres para MariaDB, la App Python y el sistema de reconocimiento facial.
- **Velocidad Estimada:** ~2.2 - 2.5 t/s (tokens por segundo) (tokens por segundo) en el EdgeBox RPi-200.

🏆 2. DeepSeek-R1-Distill-Qwen-1.5B - *La Opción de Alta Velocidad*

- **Argumentación:**

- **Razonamiento Lógico:** Fue el único modelo pequeño que resolvió perfectamente los problemas de lógica y física gracias a su proceso de "Chain of Thought".
- **Eficiencia:** Con un consumo de apenas **1.38 GB de RAM** y **3.26 t/s (tokens por segundo)**, es ideal si se detecta que el sistema se sobrecarga con el reconocimiento facial.
- **Contras:** Requiere post-procesamiento en Python para limpiar las etiquetas `<think>...</think>` antes de enviar los mensajes a Telegram, lo cual añade un paso extra de desarrollo.

🏆 3. Phi-3.5 Mini (3.8B) - *La Opción de Calidad Crítica*

- **Argumentación:**

- **Calidad de Respuesta:** Es indiscutiblemente el modelo más "inteligente" y coherente de los probados. Sus respuestas son profesionales y precisas.
- **Desventaja Fatal:** Su velocidad de **1.34 t/s (tokens por segundo)** genera una latencia alta. Un reporte de 3 párrafos podría tardar más de un minuto en generarse, lo cual degrada la experiencia del usuario en Telegram.
- **Uso:** Se recomienda como "backup" solo si los modelos anteriores fallan en la interpretación de anomalías complejas.

✗ Modelos Descartados

- **Llama 3.2 (1B y 3B):** Mostraron inestabilidad en el formato de respuesta (alucinaciones JSON) y errores graves de conocimiento físico ("el vaso se vuelve plástico"). No son confiables para un entorno industrial sin un *fine-tuning* costoso.
- **Phi-2:** Tecnología obsoleta con problemas severos de adherencia a instrucciones.

5. Conclusión Arquitectónica

Para el despliegue final en el **EdgeBox RPi-200**, se recomienda la siguiente configuración:

1. **Modelo:** Qwen 2.5 - 3B Instruct.
2. **Estrategia de Uso:** Implementación de arquitectura RAG basada en SQL mediante **function calling** manual.
3. **Gestión de Recursos:** Asignación de prioridades de CPU (`nice`) para garantizar que el reconocimiento facial (Webcam) tenga precedencia sobre la inferencia del LLM, evitando bloqueos en la interfaz de usuario (se recomienda optar por una cámara con soporte de IA como la reCamara o utilizar el EdgeBox junto con un TPU).

Aclaración crítica sobre *Function Calling* en SLMs

Es fundamental entender que **ningún Small Language Model (SLM) o TinyLLM, incluyendo Qwen 2.5 3B, posee soporte nativo para *function calling* en el sentido estricto** (es decir, con un mecanismo integrado en el motor de inferencia que interprete y ejecute automáticamente las llamadas a herramientas). A diferencia de los grandes modelos comerciales (como GPT-4 o Claude 3.5), los SLMs ejecutados en entornos como `llama.cpp` **solo generan texto**. Por lo tanto, la funcionalidad de *function calling* debe implementarse de forma **manual y robusta en el backend de la aplicación**, combinando prompting estructurado, parsing riguroso y validación de seguridad.

Utilidad del *Function Calling* en el Proyecto SIP-Edge

La implementación de este patrón es **crítica para la fiabilidad y seguridad del sistema**, por las siguientes razones:

1. **Prevención de alucinaciones:** Evita que el LLM invente valores numéricos (pesos, porcentajes de materia extraña, fechas) o hechos sobre operaciones pasadas. En su lugar, delega la obtención de datos a funciones confiables que consultan directamente la base de datos MariaDB.
2. **Acceso controlado a la base de datos:** Permite que el LLM "interactúe" con los datos sin tener acceso directo a SQL. El modelo solo puede invocar un conjunto predefinido de funciones Python (por ejemplo, `get_avg_weight_by_farm()`, `detect_anomaly_in_last_100_records()`), lo que cumple con el requisito **[RF-011]** y mitiga riesgos de inyección o consultas maliciosas.
3. **Orquestación de acciones complejas:** Facilita la creación de flujos de trabajo inteligentes, como la detección automática de anomalías (**[RF-012]**) seguida del envío de una alerta contextualizada vía Telegram (**[RF-016]**), todo orquestado por el LLM pero ejecutado por código de confianza.
4. **Cumplimiento de requisitos no funcionales:** Al limitar la generación libre del LLM a decisiones de alto nivel (qué función llamar y con qué parámetros), se mejora la predictibilidad del sistema, su rendimiento y su mantenibilidad.

Proceso de Implementación, Retos y Pruebas

La implementación exitosa de este patrón requiere un flujo de trabajo bien definido:

1. **Diseño del Prompt de Sistema:** Crear un prompt de instrucción extremadamente claro que fuerce al modelo a responder **exclusivamente** con un objeto JSON válido con las claves `function` y `params`. El prompt debe listar explícitamente las funciones disponibles y sus firmas.
2. **Parsing y Validación Robusta:** Desarrollar un parser en Python que:
 - Extraiga el bloque JSON de la salida del LLM.
 - Valide que la función solicitada esté en una lista blanca.
 - Verifique que los parámetros (`params`) tengan los tipos y rangos esperados (ej. `days` debe ser un entero positivo).

- Maneje graciosamente cualquier fallo de formato o lógica, registrando el error y devolviendo un mensaje de fallback al usuario.

3. **Ejecución Segura:** El dispatcher de funciones debe ejecutar la función solicitada en un entorno aislado, capturando cualquier excepción y transformando el resultado en un contexto que el LLM pueda usar para generar una respuesta final coherente.

Retos Principales:

- **Fiabilidad del modelo:** Aunque Qwen 2.5 es uno de los mejores SLMs en seguir instrucciones, en casos límite o con prompts ambiguos puede fallar en generar el formato correcto.
- **Latencia:** El proceso de llamada a función añade un paso adicional al flujo de interacción, lo que debe ser cuidadosamente gestionado para cumplir con **[RNF-003]** (máximo 60 segundos para reportes).

Pruebas Esenciales:

- **Pruebas de Formato:** Validar que el modelo genera consistentemente el JSON esperado bajo diversas condiciones de entrada (comandos válidos, comandos ambiguos, comandos fuera de dominio).
- **Pruebas de Frontera:** Probar con parámetros extremos (fechas futuras, IDs inexistentes) para asegurar que el parser y las funciones manejan los errores correctamente.
- **Pruebas de Estrés:** Simular múltiples solicitudes concurrentes para verificar que el sistema no colapsa y que la latencia se mantiene dentro de los límites aceptables.
- **Pruebas de Seguridad:** Intentar inyectar prompts maliciosos diseñados para hacer que el LLM genere código SQL o llame a funciones no autorizadas, verificando que el sistema las rechace.

Esta arquitectura, aunque requiere un esfuerzo de desarrollo adicional, es la única forma de aprovechar la inteligencia de un SLM en un entorno industrial crítico, garantizando al mismo tiempo la integridad, seguridad y fiabilidad de los datos.
